

Mentor Guide: Enterprise Sales Intelligence Automation System

Capgemini Internship Project — Step-by-Step Teaching Plan

Prepared for: Project Mentor | Duration: 30–45 Days | Level: Advanced Corporate

DATASET OVERVIEW (Know Before You Teach)

The student has been given a realistic business Excel file with **5 sheets**:

Sheet	Key Columns	Records
customers	customer_id, name, gender, age, city, country, email, signup_date	~250+
products	product_id, product_name, category, price, cost_price	~10+
orders	order_id, customer_id, product_id, quantity, order_date, payment_mode, region, total_amount	~300+
marketing_campaign	campaign_id, campaign_name, impressions, clicks, spend, conversions	~20+
support_tickets	ticket_id, customer_id, issue_type, priority, status, resolution_time_hours	~100+

Note for Mentor: Point out to the student that `employee_performance` table is missing from the dataset — they will need to create it manually in SQL. This is intentional and tests their initiative.

PHASE-WISE TEACHING PLAN

PHASE 1 — WEEK 1 (Days 1-7): Environment Setup & Data Understanding

Day 1-2: Project Kickoff

What to teach the student:

- Read the entire project brief first — understand all deliverables before writing a single line of code.
- Set up the environment:

Tools to Install:

- Python 3.10+ (Anaconda recommended)
- MySQL Workbench or PostgreSQL + pgAdmin
- VS Code with Python + SQL extensions
- Excel (Microsoft 365)
- Git (for version control – good corporate habit)

3. Create a project folder structure:

```
capgemini_project/  
├─ data/           → raw Excel/CSV files  
├─ sql/            → all .sql script files  
├─ python/         → all .py scripts  
├─ excel/          → dashboards  
├─ docs/           → documentation  
└─ ppt/            → final presentation
```

Mentor Talking Point: *"In a real Capgemini project, version control and folder structure are reviewed as part of code quality. Your evaluator will see if you are thinking like a professional."*

Day 3-4: Data Exploration

What the student must do:

1. Open the Excel file and study each sheet carefully.
2. Answer these questions for each table:
 - How many rows and columns?
 - Which column is the Primary Key?
 - Which columns link to other tables (Foreign Keys)?
 - Are there any missing/null values?
 - Are there any duplicate rows?
3. Draw a simple **ER Diagram** (Entity-Relationship Diagram):

```
customers (customer_id PK)
|
|--- orders (customer_id FK)
|
|--- products (product_id FK)

customers --- support_tickets (customer_id FK)
marketing_campaign (standalone)
employee_performance (to be created)
```

Key data quality issues the student should find:

- Some `gender` values are mismatched with names (e.g., "Pooja" labeled Male) → data quality issue to fix
- `total_amount` in orders may not always equal `quantity × price` → validate this
- `marketing_campaign` has `impressions < clicks` in some rows → flag as anomaly

Mentor Tip: "Always do EDA (Exploratory Data Analysis) before touching the database. You cannot design a good schema without understanding the data."

Day 5-7: SQL Database Design

Teach the student how to write CREATE TABLE statements:

```
sql
```

```
-- STEP 1: Create the database
CREATE DATABASE capgemini_sales_db;
USE capgemini_sales_db;

-- STEP 2: Create customers table
CREATE TABLE customers (
    customer_id    INT PRIMARY KEY,
    customer_name  VARCHAR(100) NOT NULL,
    gender         ENUM('Male', 'Female', 'Other'),
    age            INT CHECK (age BETWEEN 18 AND 100),
    city           VARCHAR(50),
    country        VARCHAR(50),
    email          VARCHAR(100) UNIQUE,
    signup_date    DATE
);

-- STEP 3: Create products table
CREATE TABLE products (
    product_id     INT PRIMARY KEY,
    product_name   VARCHAR(100) NOT NULL,
    category       VARCHAR(50),
    price          DECIMAL(10,2) NOT NULL,
    cost_price     DECIMAL(10,2) NOT NULL
);

-- STEP 4: Create orders table (FK references)
CREATE TABLE orders (
    order_id       INT PRIMARY KEY,
    customer_id    INT,
    product_id     INT,
    quantity       INT NOT NULL,
    order_date     DATE,
    payment_mode   VARCHAR(50),
    region         VARCHAR(50),
    total_amount   DECIMAL(12,2),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- STEP 5: Create marketing_campaign table
CREATE TABLE marketing_campaign (
    campaign_id    INT PRIMARY KEY,
    campaign_name  VARCHAR(100),
    impressions    INT,
    clicks         INT,
    spend          DECIMAL(12,2),
```

```

        conversions      INT
    );

-- STEP 6: Create support_tickets table
CREATE TABLE support_tickets (
    ticket_id             INT PRIMARY KEY,
    customer_id           INT,
    issue_type            VARCHAR(100),
    priority              ENUM('High', 'Medium', 'Low'),
    status                ENUM('Pending', 'In Progress', 'Resolved'),
    resolution_time_hours INT,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

-- STEP 7: Create employee_performance (missing from dataset – create manually)
CREATE TABLE employee_performance (
    employee_id          INT PRIMARY KEY,
    employee_name        VARCHAR(100),
    department           VARCHAR(50),
    region              VARCHAR(50),
    sales_target         DECIMAL(12,2),
    actual_sales         DECIMAL(12,2),
    quarter             VARCHAR(10),
    year                INT
);

```

Mentor Coaching Points:

- Explain **why** we use `FOREIGN KEY` — referential integrity
- Explain `CHECK` constraints — data validation at DB level
- Explain `DECIMAL(10,2)` vs `FLOAT` — financial data needs precision
- Explain creation order — parent tables before child tables

PHASE 2 — WEEK 2 (Days 8-14): Python ETL Pipeline

Day 8-9: Reading & Merging Data

Teach this Python ETL script step by step:

```
python
```

```

# Step 1: Import libraries
import pandas as pd
import numpy as np
import mysql.connector
from sqlalchemy import create_engine

# Step 2: Read all sheets from the Excel file
file_path = "data/Capgemini_Intern_Realistic_Business_Dataset.xlsx"

customers_df      = pd.read_excel(file_path, sheet_name="customers")
products_df       = pd.read_excel(file_path, sheet_name="products")
orders_df         = pd.read_excel(file_path, sheet_name="orders")
marketing_df      = pd.read_excel(file_path, sheet_name="marketing_campaign")
support_df        = pd.read_excel(file_path, sheet_name="support_tickets")

# Step 3: Quick inspection
for name, df in [("customers", customers_df), ("products", products_df),
                 ("orders", orders_df), ("marketing", marketing_df),
                 ("support", support_df)]:
    print(f"\n=== {name.upper()} ===")
    print(f"Shape: {df.shape}")
    print(df.dtypes)
    print(df.isnull().sum())

```

Interview Question to Prepare: *"What is the difference between `pd.read_csv()` and `pd.read_excel()`? What is the `sheet_name` parameter?"*

Day 10-11: Data Cleaning & Validation

Teach the student a systematic cleaning approach:

```
python
```

```

# =====
# DATA CLEANING PIPELINE
# =====

def clean_customers(df):
    """Clean the customers dataframe"""
    print(f"Before cleaning: {len(df)} rows")

    # 1. Remove exact duplicates
    df = df.drop_duplicates()

    # 2. Remove rows where customer_id is null (can't identify the record)
    df = df.dropna(subset=['customer_id'])

    # 3. Standardize gender column
    df['gender'] = df['gender'].str.strip().str.title()

    # 4. Validate email format (basic check)
    email_mask = df['email'].str.contains(r'^[\w.-]+@[\w.-]+\.\w+$', na=False, regex=True)
    invalid_emails = df[~email_mask]
    print(f"Invalid emails found: {len(invalid_emails)}")
    df = df[email_mask] # Keep only valid emails

    # 5. Validate age range
    df = df[(df['age'] >= 18) & (df['age'] <= 100)]

    # 6. Standardize dates
    df['signup_date'] = pd.to_datetime(df['signup_date'], errors='coerce')
    df = df.dropna(subset=['signup_date']) # Drop rows where date couldn't be parsed

    # 7. Strip whitespace from text columns
    str_cols = ['customer_name', 'city', 'country', 'gender']
    for col in str_cols:
        df[col] = df[col].str.strip()

    print(f"After cleaning: {len(df)} rows")
    return df

def clean_orders(df, valid_customer_ids, valid_product_ids):
    """Clean orders and validate foreign keys"""

    # 1. Remove duplicates
    df = df.drop_duplicates()

    # 2. Validate foreign keys

```

```

df = df[df['customer_id'].isin(valid_customer_ids)]
df = df[df['product_id'].isin(valid_product_ids)]

# 3. Validate business logic: quantity must be positive
df = df[df['quantity'] > 0]

# 4. Validate total_amount: flag rows where total != quantity × price (join needed)
df['order_date'] = pd.to_datetime(df['order_date'], errors='coerce')

# 5. Detect potential fraud: unusually high amounts
q99 = df['total_amount'].quantile(0.99)
df['is_suspicious'] = df['total_amount'] > q99
suspicious_count = df['is_suspicious'].sum()
print(f"Suspicious transactions (top 1%): {suspicious_count}")

return df

# Run the cleaning pipeline
customers_clean = clean_customers(customers_df)
orders_clean = clean_orders(
    orders_df,
    valid_customer_ids=set(customers_clean['customer_id']),
    valid_product_ids=set(products_df['product_id'])
)

```

Mentor Teaching Point: *"The cleaning functions should be modular — one function per table. This is called the Single Responsibility Principle. In Capgemini projects, code review will check this."*

Day 12-13: KPI Calculations in Python

```
python
```

```

# =====
# KPI CALCULATION ENGINE
# =====

def calculate_kpis(orders, products, customers, marketing):
    kpis = {}

    # Merge orders with products to get cost_price
    orders_prod = orders.merge(products[['product_id', 'cost_price', 'price']],
                                on='product_id', how='left')

    # 1. Total Revenue
    kpis['total_revenue'] = orders_prod['total_amount'].sum()

    # 2. Total Cost
    orders_prod['total_cost'] = orders_prod['quantity'] * orders_prod['cost_price']
    kpis['total_cost'] = orders_prod['total_cost'].sum()

    # 3. Profit
    kpis['profit'] = kpis['total_revenue'] - kpis['total_cost']

    # 4. Profit Margin %
    kpis['profit_margin_pct'] = (kpis['profit'] / kpis['total_revenue']) * 100

    # 5. Revenue by Region
    kpis['revenue_by_region'] = orders_prod.groupby('region')['total_amount'].sum()

    # 6. Marketing ROI
    marketing['roi'] = ((marketing['conversions'] * 5000) - marketing['spend']) / ma
    kpis['avg_roi'] = marketing['roi'].mean()

    # 7. ROAS (Return on Ad Spend)
    marketing['roas'] = (marketing['conversions'] * 5000) / marketing['spend']
    kpis['avg_roas'] = marketing['roas'].mean()

    # 8. Conversion Rate
    marketing['conversion_rate'] = marketing['conversions'] / marketing['clicks'] *
    kpis['avg_conversion_rate'] = marketing['conversion_rate'].mean()

    # 9. Churn Detection: customers who haven't ordered in 6+ months
    last_order = orders.groupby('customer_id')['order_date'].max().reset_index()
    last_order.columns = ['customer_id', 'last_order_date']
    cutoff_date = pd.Timestamp.now() - pd.DateOffset(months=6)
    inactive = last_order[last_order['last_order_date'] < cutoff_date]
    kpis['churn_count'] = len(inactive)
    kpis['churn_rate_pct'] = (len(inactive) / len(customers)) * 100

```

```

# 10. Customer Lifetime Value (simplified)
customer_revenue = orders.groupby('customer_id')['total_amount'].sum()
kpis['avg_clv'] = customer_revenue.mean()

return kpis, orders_prod, marketing

kpis, orders_enriched, marketing_enriched = calculate_kpis(
    orders_clean, products_df, customers_clean, marketing_df
)

print("\n==== KPI REPORT =====")
for k, v in kpis.items():
    if isinstance(v, float):
        print(f"{k}: {v:,.2f}")
    elif isinstance(v, pd.Series):
        print(f"\n{k}: \n{v}")
    else:
        print(f"{k}: {v}")

```

Interview Questions to Prepare:

- *"How do you calculate Churn Rate?"*
 - *"What is CLV and why does it matter for business?"*
 - *"What is ROAS and how is it different from ROI?"*
-

Day 14: Loading Data into SQL (ETL Completion)

python

```
# =====
# LOAD TO DATABASE
# =====

def load_to_database(df, table_name, engine, if_exists='replace'):
    """Load a dataframe into the SQL database"""
    try:
        df.to_sql(table_name, engine, if_exists=if_exists, index=False, chunksize=500)
        print(f"✅ Loaded {len(df)} rows into '{table_name}'")
    except Exception as e:
        print(f"❌ Error loading {table_name}: {e}")

# Create SQLAlchemy engine
engine = create_engine("mysql+mysqlconnector://root:yourpassword@localhost/capgeminidb")

# Load all tables
load_to_database(customers_clean, 'customers', engine)
load_to_database(products_df, 'products', engine)
load_to_database(orders_clean, 'orders', engine)
load_to_database(marketing_df, 'marketing_campaign', engine)
load_to_database(support_df, 'support_tickets', engine)

print("\n✅ ETL Pipeline Complete!")
```

PHASE 3 — WEEK 3 (Days 15-21): Advanced SQL

Day 15-16: JOIN Operations

Teach these JOIN queries with business context:

sql

```
-- =====
```

```
-- JOIN QUERIES WITH BUSINESS PURPOSE
```

```
-- =====
```

```
-- 1. Revenue by Customer (INNER JOIN)
```

```
SELECT
```

```
    c.customer_id,  
    c.customer_name,  
    c.city,  
    COUNT(o.order_id)          AS total_orders,  
    SUM(o.total_amount)        AS total_spent
```

```
FROM customers c
```

```
INNER JOIN orders o ON c.customer_id = o.customer_id
```

```
GROUP BY c.customer_id, c.customer_name, c.city
```

```
ORDER BY total_spent DESC;
```

```
-- 2. Product Performance Report (INNER JOIN)
```

```
SELECT
```

```
    p.product_name,  
    p.category,  
    COUNT(o.order_id)          AS orders_count,  
    SUM(o.quantity)            AS units_sold,  
    SUM(o.total_amount)        AS total_revenue,  
    SUM(o.quantity * p.cost_price) AS total_cost,  
    SUM(o.total_amount) - SUM(o.quantity * p.cost_price) AS gross_profit
```

```
FROM products p
```

```
INNER JOIN orders o ON p.product_id = o.product_id
```

```
GROUP BY p.product_id, p.product_name, p.category
```

```
ORDER BY gross_profit DESC;
```

```
-- 3. Customers with NO orders (LEFT JOIN – churn candidates)
```

```
SELECT
```

```
    c.customer_id,  
    c.customer_name,  
    c.city,  
    c.signup_date,  
    o.order_id
```

```
FROM customers c
```

```
LEFT JOIN orders o ON c.customer_id = o.customer_id
```

```
WHERE o.order_id IS NULL;
```

```
-- 4. Customer Support Analysis (JOIN)
```

```
SELECT
```

```
    c.customer_name,  
    c.city,  
    COUNT(st.ticket_id)          AS total_tickets,
```

```
    AVG(st.resolution_time_hours)      AS avg_resolution_hours,  
    SUM(CASE WHEN st.status = 'Pending' THEN 1 ELSE 0 END) AS pending_tickets  
FROM customers c  
INNER JOIN support_tickets st ON c.customer_id = st.customer_id  
GROUP BY c.customer_id, c.customer_name, c.city  
ORDER BY total_tickets DESC;
```

Day 17-18: Window Functions (High-Value SQL Skill)

sql

```

-- =====
-- WINDOW FUNCTIONS – Advanced SQL
-- =====

-- 1. Running total of revenue by date (ROW_NUMBER + SUM OVER)
SELECT
    order_date,
    total_amount,
    SUM(total_amount) OVER (ORDER BY order_date) AS running_revenue,
    ROW_NUMBER() OVER (ORDER BY total_amount DESC) AS revenue_rank
FROM orders;

-- 2. Rank customers by spending within each city
SELECT
    c.customer_name,
    c.city,
    SUM(o.total_amount) AS total_spent,
    RANK() OVER (PARTITION BY c.city ORDER BY SUM(o.total_amount) DESC) AS city_rank
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name, c.city;

-- 3. Month-over-month revenue growth
SELECT
    DATE_FORMAT(order_date, '%Y-%m') AS month,
    SUM(total_amount) AS monthly_revenue,
    LAG(SUM(total_amount)) OVER (ORDER BY DATE_FORMAT(order_date, '%Y-%m')) AS prev_
    ROUND(
        (SUM(total_amount) - LAG(SUM(total_amount)) OVER (ORDER BY DATE_FORMAT(order
        / LAG(SUM(total_amount)) OVER (ORDER BY DATE_FORMAT(order_date, '%Y-%m')) *
    ) AS growth_pct
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m');

-- 4. Top 3 products per region
SELECT * FROM (
    SELECT
        o.region,
        p.product_name,
        SUM(o.total_amount) AS revenue,
        DENSE_RANK() OVER (PARTITION BY o.region ORDER BY SUM(o.total_amount) DESC)
    FROM orders o
    JOIN products p ON o.product_id = p.product_id
    GROUP BY o.region, p.product_id, p.product_name

```

```
) ranked  
WHERE rnk <= 3;
```

Interview Questions to Prepare:

- *"Explain the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()"*
 - *"What is PARTITION BY and how is it different from GROUP BY?"*
 - *"What does LAG() do? When would you use it?"*
-

Day 19-20: Views & Stored Procedures

```
sql
```

```

-- =====
-- VIEWS
-- =====

-- Monthly KPI View
CREATE VIEW vw_monthly_kpi AS
SELECT
    DATE_FORMAT(o.order_date, '%Y-%m') AS month,
    COUNT(DISTINCT o.customer_id) AS active_customers,
    COUNT(o.order_id) AS total_orders,
    SUM(o.total_amount) AS total_revenue,
    SUM(o.quantity * p.cost_price) AS total_cost,
    SUM(o.total_amount) - SUM(o.quantity * p.cost_price) AS profit,
    ROUND((SUM(o.total_amount) - SUM(o.quantity * p.cost_price))
          / SUM(o.total_amount) * 100, 2) AS profit_margin_pct
FROM orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY DATE_FORMAT(o.order_date, '%Y-%m');

-- Regional Performance View
CREATE VIEW vw_regional_performance AS
SELECT
    region,
    COUNT(order_id) AS total_orders,
    SUM(total_amount) AS total_revenue,
    AVG(total_amount) AS avg_order_value,
    COUNT(DISTINCT customer_id) AS unique_customers
FROM orders
GROUP BY region;

-- =====
-- STORED PROCEDURES
-- =====

DELIMITER $$

-- Procedure: Monthly KPI Report for a given year/month
CREATE PROCEDURE sp_monthly_kpi_report(IN p_year INT, IN p_month INT)
BEGIN
    SELECT
        COUNT(DISTINCT customer_id) AS active_customers,
        COUNT(order_id) AS total_orders,
        SUM(total_amount) AS revenue,
        AVG(total_amount) AS avg_order_value
    FROM orders
    WHERE YEAR(order_date) = p_year

```

```
        AND MONTH(order_date) = p_month;
END$$

-- Procedure: Detect duplicate transactions
CREATE PROCEDURE sp_detect_duplicates()
BEGIN
    SELECT
        customer_id,
        product_id,
        order_date,
        total_amount,
        COUNT(*) AS duplicate_count
    FROM orders
    GROUP BY customer_id, product_id, order_date, total_amount
    HAVING COUNT(*) > 1;
END$$

DELIMITER ;

-- Call the procedures
CALL sp_monthly_kpi_report(2025, 7);
CALL sp_detect_duplicates();
```

Day 21: Business Problem Queries

sql

```
-- =====  
-- ADVANCED BUSINESS PROBLEM ANALYSIS  
-- =====
```

```
-- Problem 1: Which region had a sales drop? (Compare months)
```

```
SELECT  
    region,  
    DATE_FORMAT(order_date, '%Y-%m') AS month,  
    SUM(total_amount) AS monthly_revenue  
FROM orders  
GROUP BY region, DATE_FORMAT(order_date, '%Y-%m')  
ORDER BY region, month;
```

```
-- Problem 2: Marketing spend up but conversions down – which campaigns?
```

```
SELECT  
    campaign_name,  
    spend,  
    conversions,  
    clicks,  
    ROUND(conversions / clicks * 100, 2) AS conversion_rate_pct,  
    ROUND(spend / conversions, 2) AS cost_per_conversion  
FROM marketing_campaign  
WHERE conversions < clicks * 0.05 -- conversion rate below 5%  
ORDER BY spend DESC;
```

```
-- Problem 3: Fraud detection – transactions 3x above customer average
```

```
SELECT  
    o.order_id,  
    o.customer_id,  
    o.total_amount,  
    avg_orders.avg_amount,  
    ROUND(o.total_amount / avg_orders.avg_amount, 2) AS times_above_avg  
FROM orders o  
JOIN (  
    SELECT customer_id, AVG(total_amount) AS avg_amount  
    FROM orders  
    GROUP BY customer_id  
) avg_orders ON o.customer_id = avg_orders.customer_id  
WHERE o.total_amount > avg_orders.avg_amount * 3  
ORDER BY times_above_avg DESC;
```

```
-- Problem 4: Inactive customers (no order in 6 months) – churn candidates
```

```
SELECT  
    c.customer_id,  
    c.customer_name,  
    c.city,
```

```
c.email,  
MAX(o.order_date) AS last_order_date,  
DATEDIFF(CURDATE(), MAX(o.order_date)) AS days_inactive  
FROM customers c  
LEFT JOIN orders o ON c.customer_id = o.customer_id  
GROUP BY c.customer_id, c.customer_name, c.city, c.email  
HAVING days_inactive > 180 OR last_order_date IS NULL  
ORDER BY days_inactive DESC;
```

PHASE 4 — WEEK 4 (Days 22-28): Excel Dashboard

Day 22-23: Data Preparation for Excel

Teach the student to export clean data from Python to Excel:

```
python
```

```
import openpyxl
from openpyxl.styles import PatternFill, Font, Alignment, Border, Side
from openpyxl.chart import BarChart, LineChart, Reference
from openpyxl.chart.series import DataPoint

# Export summary data to Excel for dashboard
with pd.ExcelWriter('excel/Sales_Dashboard_Data.xlsx', engine='openpyxl') as writer:
    # Sheet 1: Raw cleaned orders
    orders_clean.to_excel(writer, sheet_name='Orders', index=False)

    # Sheet 2: Monthly KPI summary
    monthly_kpi = orders_enriched.groupby(
        orders_enriched['order_date'].dt.to_period('M')
    ).agg(
        revenue=('total_amount', 'sum'),
        orders=('order_id', 'count'),
        customers=('customer_id', 'nunique')
    ).reset_index()
    monthly_kpi.to_excel(writer, sheet_name='Monthly_KPI', index=False)

    # Sheet 3: Regional performance
    regional = orders_clean.groupby('region').agg(
        revenue=('total_amount', 'sum'),
        orders=('order_id', 'count')
    ).reset_index()
    regional.to_excel(writer, sheet_name='Regional', index=False)

    # Sheet 4: Marketing summary
    marketing_enriched.to_excel(writer, sheet_name='Marketing', index=False)

print("✅ Excel data exported successfully")
```

Day 24-26: Excel Dashboard Tasks (Teach in Excel UI)

Guide the student through each Excel task:

Task 1 — Pivot Tables

Pivot Table	Rows	Values	Filters
Revenue by Region	Region	Sum of total_amount	order_date (month)
Orders by Payment Mode	payment_mode	Count of order_id	region
Top Products	product_name	Sum of total_amount	category
Support Tickets	issue_type	Count, Avg resolution	priority

Task 2 — Key Excel Formulas to Teach

```
excel

# XLOOKUP - Get product name from product_id
=XLOOKUP(B2, Products!A:A, Products!B:B, "Not Found")

# INDEX MATCH - Find customer city by customer_id
=INDEX(Customers!E:E, MATCH(A2, Customers!A:A, 0))

# SUMIFS - Revenue for South region in Q1
=SUMIFS(Orders!H:H, Orders!G:G, "South", Orders!E:E, ">="&DATE(2025,1,1), Orders!E:E, "<="&DATE(2025,3,31))

# Profit Margin %
=((Revenue - Cost) / Revenue) * 100

# Conversion Rate
=(conversions / clicks) * 100

# ROAS
=(conversions * avg_order_value) / spend
```

Task 3 — Dashboard Layout Guide

```
+-----+
| ENTERPRISE SALES INTELLIGENCE DASHBOARD [Slicers] |
| Region: [All ▼]   Year: [2025 ▼]   Category: [All ▼] |
+-----+
| Total Revenue | Profit Margin | Total Orders | Churn Rate |
| ₹ 45,23,000 | 18.5% | 1,247 | 12.3% |
+-----+
| Revenue by Region (Bar Chart) | Monthly Trend (Line Chart)|
+-----+
| Top 5 Products (Table) | Marketing ROI (Bar Chart) |
+-----+
| Support Tickets Summary | Active vs Inactive Customers |
+-----+
```

Charts to create:

- 1. Column chart: Monthly Revenue Trend
- 2. Bar chart: Revenue by Region
- 3. Pie chart: Orders by Payment Mode
- 4. Line chart: Marketing Spend vs Conversions
- 5. Bar chart: Top 10 Customers by Revenue

PHASE 5 — WEEK 5 (Days 29-35): Automation & Reporting

Day 29-31: Automated Report Generation in Python

python

```

# =====
# AUTOMATED EXCEL REPORT GENERATOR
# =====

import openpyxl
from openpyxl.styles import PatternFill, Font, Alignment
from openpyxl.utils import get_column_letter
from datetime import datetime

def generate_kpi_report(kpis, orders_enriched, output_path):
    """Auto-generate a formatted KPI Excel report"""
    wb = openpyxl.Workbook()
    ws = wb.active
    ws.title = "KPI Report"

    # Header styles
    header_fill = PatternFill("solid", fgColor="1F4E79")
    header_font = Font(color="FFFFFF", bold=True, size=12)
    kpi_fill = PatternFill("solid", fgColor="D6E4F0")

    # Title
    ws.merge_cells("A1:F1")
    ws["A1"] = f"Enterprise Sales KPI Report – Generated {datetime.now().strftime('%Y-%m-%d %H:%M')}"
    ws["A1"].font = Font(bold=True, size=16, color="1F4E79")
    ws["A1"].alignment = Alignment(horizontal="center")

    # KPI Cards section
    ws["A3"] = "KPI"
    ws["B3"] = "Value"
    ws["A3"].fill = header_fill
    ws["A3"].font = header_font
    ws["B3"].fill = header_fill
    ws["B3"].font = header_font

    kpi_labels = {
        'total_revenue': 'Total Revenue (₹)',
        'profit': 'Total Profit (₹)',
        'profit_margin_pct': 'Profit Margin (%)',
        'avg_roi': 'Average ROI (%)',
        'avg_roas': 'Average ROAS',
        'avg_conversion_rate': 'Avg Conversion Rate (%)',
        'churn_rate_pct': 'Churn Rate (%)',
        'avg_clv': 'Avg Customer Lifetime Value (₹)',
    }

    row = 4

```

```
for key, label in kpi_labels.items():
    if key in kpis and not isinstance(kpis[key], pd.Series):
        ws[f"A{row}"] = label
        ws[f"B{row}"] = round(kpis[key], 2)
        ws[f"A{row}"].fill = kpi_fill
        row += 1

# Auto-fit columns
for col in ws.columns:
    max_len = max(len(str(cell.value or "")) for cell in col)
    ws.column_dimensions[get_column_letter(col[0].column)].width = min(max_len + 1, 100)

wb.save(output_path)
print(f"✅ KPI Report saved to {output_path}")

generate_kpi_report(kpis, orders_enriched, "excel/Automated_KPI_Report.xlsx")
```

Day 32-33: Fraud Detection Module

python

```

# =====
# FRAUD DETECTION ENGINE
# =====

def detect_fraud(orders_df):
    """Multi-rule fraud detection system"""
    fraud_flags = pd.DataFrame()
    orders_copy = orders_df.copy()

    # Rule 1: Amount 3x above customer's own average
    customer_avg = orders_copy.groupby('customer_id')['total_amount'].mean()
    orders_copy['cust_avg'] = orders_copy['customer_id'].map(customer_avg)
    orders_copy['rule1_flag'] = orders_copy['total_amount'] > (orders_copy['cust_avg'] * 3)

    # Rule 2: Same customer, same product, same day – possible duplicate
    dup_check = orders_copy.groupby(['customer_id', 'product_id', 'order_date']).size()
    dup_orders = dup_check[dup_check > 1].reset_index()
    dup_orders.columns = ['customer_id', 'product_id', 'order_date', 'count']
    dup_key = orders_copy[['customer_id', 'product_id', 'order_date']].apply(tuple, axis=1)
    dup_keys = set(zip(dup_orders.customer_id, dup_orders.product_id, dup_orders.order_date))
    orders_copy['rule2_flag'] = dup_key.isin(dup_keys)

    # Rule 3: Amount in top 1% overall
    threshold_99 = orders_copy['total_amount'].quantile(0.99)
    orders_copy['rule3_flag'] = orders_copy['total_amount'] > threshold_99

    # Combined fraud score
    orders_copy['fraud_score'] = (
        orders_copy['rule1_flag'].astype(int) +
        orders_copy['rule2_flag'].astype(int) +
        orders_copy['rule3_flag'].astype(int)
    )
    orders_copy['is_fraud'] = orders_copy['fraud_score'] >= 2

    fraud_cases = orders_copy[orders_copy['is_fraud'] == True]
    print(f"\n🚨 Fraud Detection Results:")
    print(f"Total transactions analyzed: {len(orders_copy)}")
    print(f"Suspicious transactions: {len(fraud_cases)}")
    print(f"Fraud rate: {len(fraud_cases)/len(orders_copy)*100:.2f}%")

    return orders_copy, fraud_cases

orders_scored, fraud_cases = detect_fraud(orders_clean)
fraud_cases.to_excel("excel/Fraud_Report.xlsx", index=False)

```

PHASE 6 — WEEK 6 (Days 36-42): Documentation & Presentation

Day 34-36: Documentation Writing

Teach the student what to include in project documentation:

Project Documentation Structure:

=====

1. Executive Summary (1 page)
 - What the system does
 - Business impact
 - Key findings
2. Dataset Description (1 page)
 - Table descriptions
 - Record counts
 - Data quality issues found
3. Architecture Diagram
 - Data flow: Excel → Python ETL → SQL DB → Reports
4. SQL Documentation
 - Schema design decisions
 - View and procedure descriptions
 - Sample query outputs
5. Python Code Documentation
 - Module descriptions
 - Function docstrings (student MUST add these)
 - ETL pipeline flow
6. KPI Results
 - Table of all KPIs with values
 - Insights and recommendations
7. Business Problem Analysis
 - Region sales drop: root cause findings
 - Marketing analysis: campaigns that underperformed
 - Fraud findings: count and patterns
 - Churn analysis: at-risk customer segments
8. Appendix
 - ER Diagram
 - Data dictionary

Day 37-40: PPT Presentation (10-12 Slides)

Guide the student on slide structure:

Slide	Title	Content
1	Title Slide	Project name, student name, date, Capgemini logo
2	Problem Statement	What business problems we are solving
3	Architecture	Data flow diagram (ETL pipeline)
4	Data Overview	Tables, record counts, ER diagram
5	SQL Highlights	Key JOINS, Window Functions, Stored Procedures
6	Python ETL	Cleaning steps, validation logic, automation
7	KPI Dashboard	Screenshot of Excel dashboard
8	Business Insights	4 business problems with findings
9	Fraud Detection	Method, findings, fraud rate
10	Churn Analysis	Inactive customers, recommendations
11	Challenges Faced	Data quality issues, how they were resolved
12	Conclusion	Business value delivered, skills demonstrated

INTERVIEW PREPARATION — TOP 30 QUESTIONS

SQL Questions (10 most likely)

1. What is the difference between WHERE and HAVING?
2. Explain INNER JOIN, LEFT JOIN, RIGHT JOIN with examples.
3. What are Window Functions? Give a real business use case.
4. What is the difference between RANK() and DENSE_RANK()?
5. What is a Stored Procedure? Why use it over a normal query?
6. What is a View? What are its advantages?
7. How do you detect duplicates in SQL?
8. What is a CTE (Common Table Expression)? Show an example.
9. What is the difference between DELETE, TRUNCATE, and DROP?

10. How do you calculate month-over-month growth in SQL?

Python Questions (10 most likely)

- 1. What is Pandas? How is it different from a SQL database?
- 2. What does `df.dropna()` vs `df.fillna()` do?
- 3. How do you merge two DataFrames? What are the different join types?
- 4. What is the difference between `groupby()` and `pivot_table()`?
- 5. How do you read multiple sheets from an Excel file?
- 6. What is ETL? Explain how you built the pipeline in this project.
- 7. How did you detect fraudulent transactions?
- 8. What is `apply()` in Pandas? When would you use it?
- 9. How do you export a Pandas DataFrame to Excel with formatting?
- 10. What is SQLAlchemy and why did you use it instead of just `pymysql`?

Business/KPI Questions (10 most likely)

- 1. What is Profit Margin and how is it calculated?
- 2. What is the difference between ROI and ROAS?
- 3. How do you define a customer as "churned"?
- 4. What is Customer Lifetime Value and why does it matter?
- 5. If marketing spend increased but conversions dropped, what could be the reasons?
- 6. How would you identify which region had a sales drop and why?
- 7. What is the difference between Conversion Rate and Click-Through Rate?
- 8. How do you build a KPI dashboard that is self-updating?
- 9. What is a pivot table and when would you use one in business analysis?
- 10. How would you present your findings to a non-technical business manager?

 EVALUATION SCORING GUIDE (For Mentor)

Component	Max Marks	What to Check
SQL Complexity	25	Window functions, Stored Procedures, Views, Complex JOINS
Python Logic	25	Modular code, ETL pipeline, KPI calculations, Fraud detection
Excel Dashboard	20	Pivot tables, Charts, Slicers, XLOOKUP/SUMIFS formulas
Automation	15	Python auto-generates Excel report without manual steps
Documentation	15	Code comments, project docs, ER diagram, PPT clarity

Deduct marks if:

- No docstrings or comments in Python code
- SQL queries have no business context (just technical)
- Excel dashboard has no slicers or interactivity
- Documentation is copied from the internet
- Student cannot explain their own code during viva

Award bonus marks if:

- Student creates a scheduling script (cron job / Task Scheduler) for automation
 - Student adds data validation alerts in Excel (conditional formatting)
 - Student creates an additional analysis not in the requirements
-

 MENTOR CHECKLIST — REVIEW AT EACH STAGE**End of Week 1**

- ☐ Folder structure created
- ☐ All tools installed and working
- ☐ ER diagram drawn correctly
- ☐ CREATE TABLE statements written with constraints

End of Week 2

- ☐ Python reads all 5 Excel sheets
- ☐ Cleaning functions modular and documented
- ☐ KPIs calculated correctly
- ☐ Data loaded into MySQL without errors

End of Week 3

- ☐ All JOIN queries work and return meaningful data
- ☐ Window functions implemented (RANK, LAG, SUM OVER)
- ☐ View created and queryable
- ☐ Stored Procedures created and tested

End of Week 4

- ☐ Excel data exported from Python
- ☐ At least 4 Pivot Tables created
- ☐ Dashboard layout complete with slicers
- ☐ XLOOKUP/SUMIFS formulas working

End of Week 5

- ☐ Automated report generator script works end-to-end
- ☐ Fraud detection flags suspicious rows
- ☐ All files organized in proper folders

End of Week 6

- ☐ Project documentation complete (8 sections)
- ☐ PPT has 10-12 slides with charts/screenshots
- ☐ Student can explain every line of code
- ☐ All deliverables compiled and ready

Guide prepared for Capgemini Internship Project Evaluation Version 1.0 / May 2026